

Hippocrates AI — Technical Specifications

This document outlines the technical architecture, data models, algorithms, and integration points for the Hippocrates AI platform.

1. System Architecture

Hippocrates AI uses a decoupled client-server architecture with a monolithic backend and two separate frontend clients (Web and Mobile).

1.1 High-Level Components

1. **Backend System:** Python/Django monolithic REST API handling business logic, document parsing, AI generation proxying, spaced repetition state, and PostgreSQL database interaction.
 2. **Web Client:** React 19 Single Page Application (SPA) providing the primary interface for students to upload documents, review AI-generated cards, and perform daily study sessions.
 3. **AI Engine:** Google Gemini API ([gemini-3.1-pro-preview](#)) used for semantic context extraction and structured question generation.
-

2. Technology Stack

Backend

- **Framework:** Django 4.2+, Django REST Framework (DRF) 3.15+
- **Database:** PostgreSQL (via [psycopg2-binary](#))
- **Authentication:** JWT (JSON Web Tokens via [djangorestframework-simplejwt](#))-
- **Server/WSGI:** Gunicorn + WhiteNoise for static file serving.
- **AI Integration:** [google-genai](#) and [google-generativeai](#) SDKs.

Web Frontend

- **Framework:** React 19, React Router DOM v7
 - **Build Tool:** Vite v8
 - **State Management:** Zustand (for global stores like [useStore.ts](#))
 - **Styling:** Tailwind CSS v3/v4 (PostCSS/Autoprefixer), [clsx](#), [tailwind-merge](#)
 - **Animations:** Motion (Framer Motion)
 - **Icons:** Lucide React
-

3. Core Data Architecture

The database is structured around key domains that isolate business logic and ensure scalability: Accounts, Documents, Quiz/SRS, and AI/Logging.

3.1 Document Management

- **Materials:** Represents an uploaded document. The system maintains strict immutability for source materials to ensure that generated assessments always tie back to stable reference texts.
- **Structural Blocks:** Documents are parsed and stored as hierarchical trees. This allows the system to reconstruct exact reading contexts and maintain the original layout (headings, paragraphs, lists) on the frontend.
- **Progress Tracking:** Maintains real-time data on a user's reading progression and the specific boundaries of their generated content.

3.2 Assessment & Spaced Repetition

- **Assessment Cards:** Stores AI-generated questions across multiple formats. These entities maintain relational links back to their specific source blocks for contextual grounding, and pass through a structured user-approval workflow.
- **Spaced Repetition State:** Maintains the highly dynamic, user-specific progress for each assessment card. This includes tracking intra-session performance (consecutive correct answers), lifetime mastery metrics, future scheduling dates, and precise cooldown timestamps.
- **Daily Analytics:** Aggregates user activity metrics in real-time. This provides the fast-read data layer required for dashboard visualizations, daily streak calculations, and overall progress tracking without requiring heavy, complex database queries.

4. Core Algorithms

4.1 AI Context Extraction & Prompting

To prevent the LLM from hallucinating or generating generic questions, Hippocrates AI uses **Contextual Grounding**:

1. When a user highlights text in the frontend, the system identifies the structural location of the text.
2. The backend retrieves the highlighted block, its parent heading, and adjacent text to build a dynamic context window.
3. The prompt explicitly constraints the generative model to create questions *only* from the provided context block, returning data strictly in a pre-defined JSON schema to ensure frontend compatibility.

4.2 Proprietary Spaced Repetition System (SRS)

The system departs from standard, time-only algorithms (like SM-2) by enforcing a strict **Mastery Threshold**.

1. **Intra-Session Mastery:** A card requires consecutive correct answers to achieve session mastery. Answering correctly sets a temporary cooldown before the card returns. Answering incorrectly resets the current session progress and applies a different cooldown.
2. **Mastery Threshold:** A card is only considered *Mastered* for the session when it hits a predefined streak of consecutive correct answers, eliminating lucky guesses.
3. **Long-Term Scheduling:** Once mastered, the card's lifetime mastery count increments. The system reschedules the card for future dates based on its maturity—newer concepts are reviewed sooner, while well-known concepts are pushed further into the future.

4.3 Quota-Based Queue Runner

To balance cognitive load, the backend delivers a capped batch of cards per session. These are distributed via a proprietary age-based quota system:

- **New Cards:** Concepts the user has never mastered.
- **Young Cards:** Concepts in the early stages of memory consolidation.
- **Mature Cards:** Deeply retained concepts requiring less frequent review.

Shortfall Backfilling: If a specific category cannot meet its quota, the remaining slots are dynamically filled from other categories. The backfill priority strictly favors cards the user is currently working on in the active session, preventing the loss of partial progress.

4.4 Frontend Interleaving

Instead of showing all unattempted cards followed by all review cards, the frontend uses an intelligent queue runner that interleaves cards. The algorithm blends unattempted concepts with due review cards at a specific, controlled ratio, ensuring the student is continuously challenged with new material while simultaneously reinforcing older concepts.

5. Security & Authentication

- **Authentication:** All protected endpoints require an **Authorization: Bearer <token>** header. Tokens are short-lived JWTs issued by the **accounts** Django app upon login/registration.
- **Ownership Validation:** Models (**Material**, **QuestionSchedule**) enforce user ownership. A user cannot access or review materials uploaded by another user (unless a specific sharing feature is implemented).
- **Data Protection:** **source_block** deletion is restricted (**on_delete=models.PROTECT**) to ensure questions always have their source context.

6. Deployment Infrastructure

- **Compute:** AWS EC2 instances managed by an Auto Scaling Group, sitting behind an Application Load Balancer (ALB) to handle traffic spikes during exam periods.
- **Database:** Amazon RDS (PostgreSQL) in a Multi-AZ deployment for high availability.
- **Storage:** Amazon S3 for storing raw **Material** files (**.pdf**, **.docx**).
- **CDN:** CloudFront (or WhiteNoise natively) for caching static assets.
- **AI Gateway:** Requests to Google Gemini are made server-side (Django) to protect API keys and ensure all context extraction logic remains secure and un-manipulated by the client.